Week 09: TESTING

G1: Anchor Assertions To Spec. (Not Code)

- Supply spec. as basis for expected values — don't let model derive them solely from reading the implementation
- A model reading only code repeats its bugs as expected — oracle deficiency; top practitioner concern
- Give docstring / spec / worked example; instruct model to "derive from spec, not by tracing code"
- Numeric → Give formula; strings → give reference doc example; not implementation
- Ask "could this be read off the implementation" → verify v/s spec; test against known-buggy version (pass = red flag)

G2: Boundary / Equivalence - Class Coverage

- List boundary conditions / equivalence classes explicitly in the prompt; don't assume model applies structured design on its own
- Reasoning: LLMs default to happy-path; branch coverage remains weakest dimension without guidance
- Prompting: List classes / boundaries yourself first, paste verbatim into prompt
- Prompting: Numeric → name exact values (0, 1, 99, 100, 101); strings → empty; mid-len, max-len, invalid char
- Review: Boundaries tested at the edge itself; invalid-class tests assert rejection; not just "no crash"

G3: Specify Mocking / Stubbing

- Explicitly name each dependency and specify mock / stub — don't let model decide isolation on its own
- Reasoning: LLMs unreliable alone — skip mocking, use unrealistic mocks, or mock wrong boundary
- Prompting: name dependency + framework + injection point; describe stub behavior per variant
- Prompting: Ask model to assert on mock interactions (call count / args)
- Review: No real network / DB / file calls; mock sits at close edge with realistic return values

G4: Properties / Metamorphic Relations

- For hard-to-derive outputs, prompt for property-based tests / metamorphic relations, not concrete assertions
- Reasoning: sidesteps oracle problem — express what's always true rather than exact output
- Prompting: Explicitly request property-based tests + name framework; have model enumerate invariants
- Prompting: Use categories — round-trip, idempotency, monotonicity, metamorphic ("f(x) v/s f(y)")
- Review: run 500-1000+ samples, ensure edge cases generated, check properties aren't vacuous

G5: Regression Tests from Bug Reports / Diffs

- Prompt with bug description or diff, ask for test that would've caught it — write before / alongside fix
- Reasoning: must fail pre-fix, pass post-fix — general coverage-driven generation won't fix this
- Feed bug report / diff verbatim (repro steps + observed v/s expected); instruct "must FAIL before fix"
- Ask for comment explaining why test catches the bug — confirms correct understanding
- Verify fails pre-fix checkout, passes post-fix, narrow to the specific input

G6: Mutation Testing > Coverage Alone

- Don't use line coverage as sole quality signal — use mutation testing to confirm suite detects fault
- Reasoning: AI assertions often syntactically fine but semantically weak — coverage ≠ fault detection
- Prompting: Target specific mutations you list (eg: > → >=); annotate assertions with fault they catch
- Prompting: Numeric → request tests at off-by-one sensitive values
- Review: Run mutmut / PIT, feed surviving mutants back for targeted tests; score $\frac{\text{mutants killed}}{\text{total mutants}}$ ≤ 70% indicates "write more"; ignore 100% coverage claims alone [mutmut: Python; PITest: Java]

WEEK-10: Code Review

G2: Prepare Small, Well-described Changes

- Keep each PR to one logical unit; write clear descriptions before AI review
- Small reviews (200 LOC/hr) catch ~2/3 design defects; LLM precision < 10% on large diffs.
- Prompting: One logical change per PR — split refactor / rename / feature into separate PRs
- Prompting: If diff unreadable in 30 min, split into stacked PRs; ask AI to suggest commit order
- Include what / why / how tested / what to check; name invariants to preserve; state rejected alternatives

G2: Feed the Reviewer the Surrounding Code

- Send enclosing function / class + imports + call site; not just changed lines
- Reasoning: Insufficient context → false positives; AST-boundary context yields more reliable comments
- Prompting: Paste full enclosing function / class; mark challenged lines (+/-)
- Prompting: Include imports / type defs; for public functions, add at least one call site
- Flag AI comments referencing code not provided (guesses); re-run after adding missing context

G3: Run the Review as Two Passes

- Seperate comprehension pass (explain the change) from rubric pass (evaluate against criteria)
- Defect detection is downstream of comprehension — wrong summary poisons all critique
- Pass 1: Ask AI to describe change in 3-5 sentences (no bugs/issues); infer intent + invariants
- Pass 2: Short imperative rules under headings (security / perf / error handling / style); scoped & specific
- Read summary first; fix prompt if wrong before rubric pass; include negative rules; store rubric in version control

G4: Treat Findings As Hypothesis

- Treat every AI comment as an unverified claim, not a verdict
- LLM precision < 10% on full-PR reviews; priming with prior comments causes reviewers to over-anchor
- Prompting: Require each finding to cite exact lines + evidence; ask for minimal failing example
- Review: Locate evidence yourself. No evidence = suspect; classify as confirmed / style / false-positive
- Never batch-accept "apply all"; for second opinion, run blind first, then a primed consolidation pass - only

G5: Own the Merge, Own the Learning

- Human keeps merge decision, approval & accountability; AI can't own these
- Reasoning: Cannot be accountable to an LLM; quality drivers are interpersonal; automation bias causes mixed errors
- Ask for evidence-based observations, not verdicts; ask AI to rank confidence & flag guesses
- Ask for alternative approaches, not just defects; use AI's "why it matters" as raw material
- Read the diff yourself first; write comments that explain why; respond to author directly

G6: Heightened Scrutiny for AI-authored / Security-sensitive code

- Default AI review calibrated for routine code — security sensitive / AI authored changes need more
- AI authored PRs ~1.7x more issues; malicious-PR detectors hit 99.3% with static analysis + named threat list
- PR mentions AI co-authorship, touches auth/crypto/IO/CI, first-time contributor or unusually large/broad
- Give AI a named threat list rather than a vague "is this safe?"
- Run static analysis / dependency scanners first; ask AI to verify every external symbol exists;
  human always reads auth / crypto / subprocess / CI changes regardless of tool output

WEEK-11 : PERFORMANCE

G1: Feed Profiler Output Back Into Prompt

- Run a profiler first; give the model the report; not just "optimize this"
- Profile-guided self-optimization loops cut runtime ~87%, memory ~91% w/ negligible correctness loss
- Prompting: Paste top 10-20 lines of cProfile / perf / gprof / Valgrind output
- State bottleneck explicitly - function name, % runtime, call count, input size / access pattern
- Review: Re-run profiler after; reject rewrite unless bottleneck's share visibly dropped

G2: Verify Algorithmic Complexity of Generated Code

- Never assume correct output is also efficient - demand complexity analysis
- Reasoning: GPT-4 pass@1 0.831 v/s eff@1 0.454 - half of correct sols. aren't runtime-competitive
- Prompting: Specify target complexity upfront (Eg: "O(nlogn) time, O(n) space")
- Ask model to identify best-known complexity before coding, then justify after
- Check hidden O(n) ops; empirically test n = 1000, 2000, 4000 scaling

YASH BARVE
yhbarve
20930353

G3: Provide Input Characteristics to Prompt Algorithm Switching

- Give typical input distribution so model can build hybrid fast / slow-path logic
- Models default to one general algorithm; won't hybridize without explicit distribution info.
- State input size frequency (eg: "90% n < 32; 10% n up to 100K") — crossover point
- Prompting: Request named branches (_small_case / _general_case) with per-branch complexity +
- Test boundary cases (threshold - 1, threshold, threshold + 1) & measure actual empirical crossover

G4: Provide a Fast Reference Implementation

- Give a known-fast example / pattern; frame task as slow → fast implementation
- PIE-prompted models hit 6.86x mean speed up v/s laziest-default transforms otherwise
- Supply SLOW + FAST reference pair explicitly labeled, using desired strategy (DP/hash)
- Ask model to narrate mapping between reference & new implementation — can't be a constant
- Confirm transformation actually applied (not verbatim copy); speedup should scale with n

G5: Specify Memory Hierarchy Constraints for Low-Level Optimization

- State cache size, data type, access pattern before requesting C++ loop optimization
- Without this, models produce cache-hostile, readability-optimized code
- Name cache-size, line-width, and data footprint
- For SIMD, restrict to standard intrinsics / OpenMP programs and require justification per intrinsic
- Verify with 'perf stat -e cache-misses', and check compiled binary actually vectorizes

G6: Audit Parallelism Proposals for Correctness Before Measuring Speedup

- Treat AI-added parallelism as a correctness-critical draft, not a finished optimization
- Parallel pass@1 only 37.8 v/s 84.1 serial - races / missing syncs are common silent failures
- Name shared mutable state explicitly & required sync strategy
- For Python, clarify GIL (Global Interpretor Lock) - ProcessPoolExecutor for CPU-Bound, ThreadPoolExecutor for I/O-Bound
- Run parallel v/s serial on deterministic input ~10x to catch non-deterministic races; check hardcoded thread counts

WEEK-12: SECURITY

G1: Enumerate Candidate Threat Models Before Writing Code

- Have AI list applicable threats w/ CWE IDs + reasoning before generating security-sensitive code
- LLMs miss context-dependent flaws (eg: broken access control) invisible from code alone
- Describe system plainly; request Threat / CWE ID / Reason table
- Use STRIDE categories; ask for plausible threats, not a generic checklist
- Treat list as candidate - verify CWE IDs, add omitted threats, carry confirmed list forward

G2: Declare the Threat Model in Every Security-Sensitive Prompt

- Name specific vulnerabilities to defend against + exact libraries / standards in the prompt itself
- Omitting constraints yields functionally correct code that silently skips defenses
- Paste confirmed threat list as numbered requirements; name each vulnerability by CWE ID
- Specify exact mechanism; include full contract in one prompt, not follow-up
- Confirm every named defense appears in output, check error messages don't leak internal info.

G3: Generate Exploit-Style Tests From the Threat Model

- After hardening the code, have the AI write one test per CWE from the threat model
- Unanchored "make it more secure" iteration raises critical vulnerabilities by 37.6%.
- Paste threat model table alongside code; one test per CWE, labeled by ID
- Each test = exploit attempt + assertion; use Hypothesis for free-form string inputs
- Confirm test fails on naive / passes on hardened; assertions go beyond status codes

G4: Prompt For Hardened Infrastructure Configs & Verify Before Development

- Explicitly name security constraints when generating Docker / DB / cloud configs; verify before deploying
- AI generated IAC defaults to open ports / wildcard IAM / public resources; passes app-level SAST checks
- State deployment environment + constraints; Ask model to justify every exposed port / permission
- Check bind addresses; auth / default creds replaced, IAM wildcards / open CIDRs

G5: Verify Every AI-Suggested Dependency Before Installing

- Manually confirm any AI-suggested package exists on official registering before installing.
- "Slopsquatting": Attackers register packages matching names AI commonly hallucinates — confirmation
- Require well-known packages (1M+ monthly downloads); ask for pinned versions & existence
- If unsure, ask model directly "does this package exist on PyPI/npm?"
- Search registry, run pip/npm audit post-install; enable Dependabot/Snyk; pin CVE-free versions

G6: Anchor LLM Vulnerability Review with SAST Tool Output

- Run static analysis first, paste its output into the prompt instead of asking AI to "review for security issues"
- CodeQL-grounded prompts cut residual vulnerability rate
- Run Bandit / Semgrep / CodeQL first; paste full output verbatim in the prompt
- Ask model to explain why flagged line is vulnerable before requesting fix; max 2 iteration rounds — w/o human checkpoint
- Re-run same SAST Tool on the fix; check no new vulnerability is introduced; confirm fix is logically equivalent to original

SAST: Static Application Security Testing